

Group Project Proposal

AI Researcher Agent:
An Auditable Workflow from Ideas to Experiments and Papers

Group ID: _____ 6 _____

DSA4213: Natural Language Processing for Data Science
Semester 1, AY2026/2027

Administrative Information

Group members	DENG DELIN (A0355114M) GUO RUI (A0351598N) PENG LIJIN (A0355127E) ZHANG YUEQIAN (A0351049H) ZHU YANG (A0355627W)
Project direction	Autonomous research in computer science.
Repository	https://github.com/sunnyoung0211/ai-researcher-agent

Project Summary

We propose an AI Researcher Agent that helps users develop research ideas, conduct computational experiments, and produce a LaTeX manuscript. The system will accept either a user-provided idea or a research direction, retrieve and analyse relevant literature, archive an approved research objective, and execute a bounded experiment loop locally or on an SSH server. It will then generate figures and a manuscript using a user-uploaded template. Users will approve ideas and their revisions, experimental plans, important research-log entries, and the final manuscript.

Our central question is whether versioned research records and explicit claim-to-evidence checks improve the reliability of an agent’s research outputs under a fixed resource budget. We will implement a persistent workflow connecting decisions, code, environments, runs, figures, and manuscript claims. Evaluation will combine an end-to-end demonstration, capability ablations, and reproducibility checks. A possible demonstration is in NLP, but the workflow will support configurable tasks across computer science rather than require a particular dataset, model, or training pipeline.

1. Problem, Research Question, and Motivation

Computational research requires more than producing plausible text or executable code. A researcher must refine a question, justify comparisons, interpret unexpected results, and ensure that the final report accurately represents the experiments. When an agent moves between these activities, it may lose constraints, reuse outdated results, or make claims unsupported by its recorded evidence. These failures are difficult to detect from a polished manuscript alone.

Our intended users are students and researchers conducting computational studies. They should be able to inspect why an experiment was proposed, recover an interrupted project, and trace a

result back to its code and execution conditions. An agent-based approach is appropriate because the next action depends on observations: a failed run may require debugging, an unstable result may motivate replication, and an infeasible idea may require a revised plan.

Research question. Under a fixed computation and model-call budget, does an agent with versioned research records and claim-to-evidence checks produce fewer unsupported claims and more reproducible experiment packages than the same workflow without those checks?

Success criteria. We aim to complete an approved idea-to-manuscript workflow, run a baseline and a candidate method with a relevant control, recover execution state, and allow another person to reproduce a core result from the archive. The evaluation must establish whether the proposed checks help, including their costs and failure modes. A negative scientific result or an inconclusive system comparison remains a valid outcome; producing a publishable discovery is not a success requirement.

2. Related Work and Proposed Contribution

The AI Scientist generates ideas, writes and executes experiment code, visualises results, and writes and reviews scientific papers [1]. It provides the closest precedent for our three-stage workflow. SWE-agent studies interfaces through which language-model agents navigate repositories, edit code, and run tests and programs [2]. It motivates treating execution feedback as an observation that informs the next action. These systems establish that tool-using agents can perform substantial research and programming activities; simply connecting a literature tool, a coding agent, and a writer would therefore be an insufficient contribution.

Our proposed contribution is an **inspectable research workflow with evidence checks and version-specific human approval**. We will implement the linkage between approved objectives, experiment plans, execution records, generated figures, and manuscript claims, and study whether this linkage improves reliability. A claim should identify the supporting run, metric, evaluation condition, and aggregation, not merely point to an experiment directory. We will also preserve rejected ideas, failed runs, and changes in interpretation.

A secondary capability is task-specific, low-cost multi-agent literature work: agents will search subtopics and read papers from different perspectives, while a coordinator merges evidence and exposes disagreements. We will evaluate this capability where resources permit rather than assume parallel agents necessarily improve quality. Existing models, retrieval services, execution tools, and LaTeX utilities may be reused; our work centres on workflow control, persistent evidence, approval behaviour, and empirical evaluation.

3. Agent and Task Setup

Task and environment. The agent will conduct code-based experiments in computer science, including potentially NLP, algorithms, systems, and software engineering. Projects may use datasets, workloads, test cases, or simulators; a trainable model is optional. The first implementation will support a controlled toolchain and both local execution and user-configured SSH servers. Environment snapshots will record code and input versions, dependencies, operating system, hardware, launch commands, and random seeds where relevant.

Inputs and outputs. Inputs include an idea or domain, resource constraints, accessible research materials, execution settings, and an optional LaTeX template. Outputs include candidate ideas and literature notes, an approved research objective, versioned plans, code and run records, figures, a compiled manuscript, and an indexed research archive. The specific demonstration task, dataset or workload, model/system scale, model provider, and numerical budget remain open selections; we do not prescribe them in this proposal.

Models, tools, and memory. A main agent will manage the research process. Multiple literature agents will preferentially use low-cost, fast models with prompts for search, method extraction, research-gap analysis, and feasibility assessment. Tools will support retrieval, file operations, code checking, execution, metric aggregation, plotting, and LaTeX compilation. Paper sources and their APIs, search strategies, fees, and rate limits will be selected during implementation. Retrieved material will retain its source, access date, and available evidence; an abstract-only reading will be labelled accordingly. Persistent project files and event records, rather than conversation history alone, will provide memory.

User control and stopping. Approval will be attached to specific versions of ideas, plans, important research-log entries, and the final manuscript. Important entries include major interpretations, result-validity decisions, and abandoned research routes; raw outputs and error logs will be stored automatically. Within an approved plan and budget, the agent may execute tasks and perform bounded debugging without repeated approval. A new or revised plan requires approval even if budget remains. Execution stops on completion, budget exhaustion, a configured iteration/retry limit, cancellation, or an unrecoverable failure. An SSH disconnection will trigger status reconciliation before any retry, preventing duplicate jobs.

4. Proposed System or Study Design

The system will use a shared backend for orchestration, persistent state, approvals, and artifacts. Its principal stages are:

1. **Idea development and archiving.** Refine a supplied idea or propose candidates within a domain. Retrieve literature, distribute reading tasks, compare evidence, and assess feasibility and potential novelty. Archive the user-approved question, hypothesis, scope, sources, and constraints. Retrieval can suggest novelty but cannot guarantee it.
2. **Experiment planning and execution.** Translate the hypothesis into baselines, variables, controls, evaluation conditions, and stopping rules. After plan approval, generate or modify code, perform a small trial, and execute locally or remotely. Assign each run an identifier and retain its code, configuration, environment, logs, metrics, and status. Analyse results and either continue within the approved plan, request a revised plan, or stop. Preserve unsuccessful and negative results.
3. **Evidence-grounded writing.** Construct a claim-to-evidence record and generate tables and plots from stored results. Use reusable skills for scientific plotting and LaTeX writing; each skill provides task instructions, templates or scripts, and validation rules. Compile against the user’s uploaded template, check references and numerical consistency, and submit the manuscript for

approval. The archive will support navigation from a claim to a figure, source runs, code, and configuration.

Development and interface. We will first validate a minimal CLI workflow, including persistent approval state, a small experiment, and manuscript generation. Once this works, we will add a lightweight browser GUI backed by a local service, without waiting for every research capability to be complete. The final product will retain both interfaces. The GUI will expose project status, approval queues and version differences, template upload, execution controls, logs, figures, and manuscript previews. Closing a client will not implicitly cancel a backend job; a restarted backend will reconcile saved state with actual local or remote execution.

Scope of the first delivery. We will prioritise one bounded end-to-end study, local and SSH operation, literature-agent collaboration, the four approval categories, two writing/plotting skills, and evidence-linked output. A lightweight non-NLP adaptation check will examine whether new task inputs and metrics can be introduced without rewriting the workflow. This demonstrates adaptability of the workflow, not research competence across all of computer science. Large-scale pretraining, physical experiments, automatic publication, and a general-purpose visual workflow editor are outside the initial scope.

4.1 Comparisons and Ablations (if applicable)

The primary comparison will use the same agent workflow *with and without explicit claim-to-evidence checks*. Both conditions will retain raw execution logs for independent assessment and use the same task inputs, model-role configuration, tools, initial experiment setup, approval rules, and total budget. This isolates the effect of the evidence-checking component rather than attributing all benefits of the complete product to one mechanism. Persistence and recovery will also be evaluated through targeted interruption tests.

A secondary comparison, subject to budget, will contrast a single literature agent with multiple task-specific literature agents using the same source snapshot and a matched total model-call budget. We will report coverage, summary accuracy, latency, and cost. User approval requirements will remain in all conditions; disabling an automatic checker will not remove human control.

5. Preliminary Evaluation Plan

We will give equal attention to **end-to-end experience, agent capability, and rigorous empirical comparison** (1:1:1 in our internal evaluation priorities). We will report these dimensions separately, supported by the following evidence.

Objective	Measures	Evidence / procedure
End-to-end usability	Completion rate; user interventions; correct approval and recovery behaviour	GUI demonstration from idea to approved paper; local/SSH checks; close/reopen and stale-approval scenarios.
Research reliability	Unsupported-claim rate; numerical consistency; evidence coverage	Independent inspection of manuscripts against original run records and cited literature.
Reproducibility	Successful core reruns; metadata completeness; agreement within tolerance	Re-execution in a compatible clean environment by a member who did not implement the workflow.
Literature capability	Relevant-paper coverage; factual summary accuracy; disagreements resolved	Review against a curated set of task-relevant sources; single-agent comparison if feasible.
Efficiency	Model usage; CPU/GPU time; elapsed time; failure and retry counts	Complete trajectories, including failed, timed-out, and negative-result runs.

Protocol. After a cost pilot, we propose three bounded problem instances from one task family, each repeated for three agent trajectories per primary condition (18 trajectories in total). This is a provisional scale, to be adjusted before the main study if necessary. NLP is a possible main task family; the final task and assets will be selected later. We will freeze evaluation inputs, source snapshots, model/prompt versions, budgets, and decision rules before comparison. Human decisions will follow the same rubric across conditions and all manual edits will be recorded. Agent-trajectory repetition will be distinguished from repeated measurements or random seeds inside a scientific experiment.

Claim assessment. Two members will independently review anonymised, shuffled outputs using a fixed rubric, then reconcile disagreements. Unsupported-claim rate is the number of unsupported or contradicted major claims divided by all assessed major claims. Numerical consistency is the proportion of checked numbers that match their records. We will also measure question coverage and report completeness so that an empty or evasive report cannot achieve a misleadingly good score. Outputs containing no assessable claims will be reported separately as incomplete, not assigned zero error. LLM review may assist but will not be the sole factual judge.

Reproducibility and uncertainty. Core results will be rerun with tolerances defined before inspection; nondeterministic results will include their variation. For learning tasks, exploration will use training/validation data rather than the final test set; analogous separation will apply to workloads and test cases in other domains. We will report per-instance outcomes and descriptive variability, avoiding broad significance or generalisation claims from a small sample. Human waiting time will be separated from computation time.

Failure analysis. We will include representative successes and failures and test compilation errors, inaccessible papers, interrupted SSH connections, exhausted budgets, and rejected approvals. The completion-rate denominator will include all launched tasks. A lightweight non-NLP case will test configuration-level adaptability, while a second execution environment will test portability. Neither is evidence of universal capability.

6. Risks, Limitations, and Responsible Practice

- **False evidence and overinterpretation.** Preserve source locations, verify references and numerical claims, and distinguish observed results from explanations. Human approval does not by itself establish scientific validity. Rejected interpretations remain versioned; original data will not be rewritten.
- **Execution and external-content risks.** Restrict generated code to approved environments and directories. Treat papers, webpages, repository text, and uploaded templates as untrusted inputs. Keep credentials out of archives and manuscripts, restrict template compilation, and require appropriate authorisation for external publication or data transfer beyond configured local/SSH operation.
- **Resource and integration limits.** Use small trial runs, hard budgets, bounded retries, and checkpoints. Limit the initial toolchain and template compatibility, expose unsupported dependencies, and record inaccessible remote artifacts instead of claiming full reproducibility.
- **Evaluation limits.** Agent stochasticity, human judgement, model changes, and a small task set may affect conclusions. Freeze configurations, report reviewer disagreements, and retain failures and negative results. A workflow adaptation check will not establish broad scientific generalisation.
- **Research integrity.** Respect dataset, code, and paper access permissions; record licences where relevant. Disclose the agent’s role and human review in generated manuscripts. The system will not guarantee novelty, publication acceptance, or superior scientific results, and manuscript approval will not authorise automatic submission.

References

- [1] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024. <https://arxiv.org/abs/2408.06292>.
- [2] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024. <https://arxiv.org/abs/2405.15793>.